

Sắp xếp (Sorting)

Nội dung

1. Tại sao sắp xếp quan trọng?
2. Sử dụng `sort()`
 - 2.1. Sắp xếp tăng dần (mặc định)
 - 2.2. Sắp xếp giảm dần
 - 2.3. Sắp xếp một phần mảng
 - 2.4. Sắp xếp mảng / danh sách thông thường
3. Custom Comparator (Sắp xếp theo tiêu chí riêng)
 - 3.1. C++
 - Dùng hàm (function)
 - Dùng lambda (viết gọn hơn)
 - Dùng `operator<` (overload toán tử)
 - 3.2. Python
 - Dùng `key=`
 - Với dictionary / object
 - Lưu ý quan trọng
4. Sắp xếp với pair, tuple
 - 4.1. Sắp xếp pair / tuple
 - 4.2. Mẹo: Sắp xếp "ngược" một thành phần
 - 4.3. Sắp xếp theo chỉ số (index sort)
5. Counting Sort (Sắp xếp đếm)
 - Ý tưởng
 - Ví dụ trực quan
 - Code
 - Khi nào dùng counting sort?
6. Stable Sort (Sắp xếp ổn định)
7. Loại bỏ phần tử trùng lặp
8. Ví dụ chi tiết: Bài "Chênh lệch nhỏ nhất" (Closest Pair)
 - Phân tích
 - Chạy tay
 - Code
9. Các lỗi phổ biến khi sắp xếp

- Ghi chú Python: `sorted()` vs `.sort()`
- Tóm tắt bài học

Sắp xếp là một trong những công cụ nền tảng và quan trọng nhất trong lập trình thi đấu. Bạn sẽ ngạc nhiên khi biết rằng rất nhiều bài toán tưởng phức tạp, nhưng chỉ cần sắp xếp dữ liệu là trở nên đơn giản hơn rất nhiều!

Trong bài học này, chúng ta sẽ tìm hiểu cách dùng hàm `sort()` trong C++ và Python, cách sắp xếp theo tiêu chí riêng, và các ứng dụng phổ biến của sắp xếp trong CP.

1. Tại sao sắp xếp quan trọng?

Hãy xem một ví dụ đơn giản. Cho mảng $a = [5, 2, 8, 1, 9, 3]$. Hỏi: "Hai số gần nhau nhất chênh lệch bao nhiêu?"

- **Không sort:** phải so sánh mọi cặp $\rightarrow O(n^2)$
- **Có sort:** sau khi sort ta được $[1, 2, 3, 5, 8, 9]$. Hai số gần nhau nhất chắc chắn nằm kề nhau trong mảng đã sort $\rightarrow$ chỉ cần duyệt 1 lần $\rightarrow O(n \log n)$

Đó là sức mạnh của sắp xếp: biến bài toán phức tạp thành đơn giản!

2. Sử dụng `sort()`

Cả C++ và Python đều có hàm sắp xếp rất mạnh, thường đủ dùng cho hầu hết các bài CP với độ phức tạp $O(n \log n)$.

- C++: dùng `sort(begin, end)`
- Python: dùng `a.sort()` hoặc `sorted(a)`

2.1. Sắp xếp tăng dần (mặc định)

C++

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n;
    cin >> n;
    vector<int> a(n);
    for (int &x : a) cin >> x;
    sort(a.begin(), a.end()); // Sắp xếp tăng dần
    for (int x : a) cout << x << " ";
    cout << "\n";
}
```

Python

```
n = int(input())
a = list(map(int, input().split()))
a.sort() # Sắp xếp tăng dần
print(*a)
```

2.2. Sắp xếp giảm dần

C++

```
// Cách 1: Dùng greater<int>()
sort(a.begin(), a.end(), greater<int>());

// Cách 2: Sort tăng rồi reverse
sort(a.begin(), a.end());
reverse(a.begin(), a.end());
```

Python

```
# Cách 1: Dùng reverse=True
a.sort(reverse=True)

# Cách 2: sorted() trả về list mới
b = sorted(a, reverse=True)
```

2.3. Sắp xếp một phần mảng

C++

```
// Sort từ vị trí 2 đến vị trí 5 (không bao gồm 5)
sort(a.begin() + 2, a.begin() + 5);
// Ví dụ: a = [5, 2, 8, 1, 9, 3]
// Sau khi sort vị trí 2-4: a = [5, 2, 1, 8, 9, 3]
```

Python

```
# Python không có sort trực tiếp trên đoạn con như C++
# Ta có thể gán lại slice đã được sort
a[2:5] = sorted(a[2:5])
# Ví dụ: a = [5, 2, 8, 1, 9, 3]
# Sau khi sort a[2:5]: a = [5, 2, 1, 8, 9, 3]
```

2.4. Sắp xếp mảng / danh sách thông thường

C++

```
int a[] = {5, 2, 8, 1, 9, 3};
int n = 6;
```

```
sort(a, a + n); // Sort toàn bộ mảng
```

Python

```
a = [5, 2, 8, 1, 9, 3]
a.sort()
```

3. Custom Comparator (Sắp xếp theo tiêu chí riêng)

Mặc định `sort()` sắp xếp tăng dần. Nhưng nếu bạn muốn sắp xếp theo cách khác (ví dụ: theo điểm giảm dần, nếu cùng điểm thì theo tên tăng dần), bạn cần viết tiêu chí sắp xếp riêng.

3.1. C++

Dùng hàm (function)

```
struct Student {
    string name;
    int score;
};

// Hàm so sánh: trả về true nếu a nên đứng TRƯỚC b
bool compare(const Student &a, const Student &b) {
    if (a.score != b.score) return a.score > b.score; // điểm giảm dần
    return a.name < b.name; // tên tăng dần
}

int main() {
    vector<Student> v = {"An", 85}, {"Binh", 92}, {"Cuc", 85}, {"Dung", 92}};
    sort(v.begin(), v.end(), compare);
    // Kết quả: Binh(92), Dung(92), An(85), Cuc(85)
}
```

Dùng lambda (viết gọn hơn)

```
sort(v.begin(), v.end(), [](const Student &a, const Student &b) {
    if (a.score != b.score) return a.score > b.score;
    return a.name < b.name;
});
```

Dùng operator< (overload toán tử)

```
struct Student {
    string name;
    int score;
    bool operator<(const Student &other) const {
        if (score != other.score) return score > other.score;
```

```

        return name < other.name;
    }
};

// Giờ chỉ cần:
sort(v.begin(), v.end()); // Tự dùng operator<

```

3.2. Python

Python không dùng comparator kiểu C++ trong `sort()` thông thường. Thay vào đó, ta dùng `key=`.

Dùng `key=`

```

students = [("An", 85), ("Binh", 92), ("Cuc", 85), ("Dung", 92)]
# Sort theo điểm giảm dần, nếu bằng nhau thì tên tăng dần
students.sort(key=lambda s: (-s[1], s[0]))
# Kết quả:
# [("Binh", 92), ("Dung", 92), ("An", 85), ("Cuc", 85)]

```

Với dictionary / object

```

students = [
    {"name": "An", "score": 85},
    {"name": "Binh", "score": 92},
    {"name": "Cuc", "score": 85},
    {"name": "Dung", "score": 92},
]
students.sort(key=lambda s: (-s["score"], s["name"]))

```

Lưu ý quan trọng:

- C++ thường dùng comparator
- Python thường dùng key function
- key trong Python rất mạnh và thường dễ viết hơn comparator

4. Sắp xếp với pair, tuple

pair trong C++ và tuple trong Python/C++ đều tự động so sánh theo thứ tự các thành phần (lexicographic order). Đây là kỹ thuật RẤT hữu ích để tránh viết comparator phức tạp.

4.1. Sắp xếp pair / tuple

C++

```
vector<pair<int, string>> v = {{85, "An"}, {92, "Binh"}, {85, "Cuc"}};
sort(v.begin(), v.end());
// Kết quả: (85, "An"), (85, "Cuc"), (92, "Binh")
// Sort theo first trước, nếu bằng thì theo second
```

Python

```
v = [(85, "An"), (92, "Binh"), (85, "Cuc")]
v.sort()
# Kết quả: (85, "An"), (85, "Cuc"), (92, "Binh")
```

4.2. Mẹo: Sắp xếp "ngược" một thành phần

Nếu muốn sort theo điểm giảm dần, tên tăng dần, dùng mẹo đảo dấu:

C++

```
vector<pair<int, string>> v;
for (auto &s : students) {
    v.push_back({-s.score, s.name}); // Đảo dấu điểm!
}
sort(v.begin(), v.end());
```

Python

```
students = [("An", 85), ("Binh", 92), ("Cuc", 85)]
v = [(-score, name) for name, score in students]
v.sort()
```

4.3. Sắp xếp theo chỉ số (index sort)

Đôi khi bạn muốn biết "sau khi sort, phần tử gốc ở vị trí nào":

C++

```
int a[] = {50, 20, 80, 10, 40};
int n = 5;
vector<int> idx(n);
iota(idx.begin(), idx.end(), 0); // idx = {0, 1, 2, 3, 4}
sort(idx.begin(), idx.end(), [&](int i, int j) {
    return a[i] < a[j];
});
// idx = {3, 1, 4, 0, 2}
```

Python

```
a = [50, 20, 80, 10, 40]
idx = list(range(len(a)))
idx.sort(key=lambda i: a[i])
```

```
# idx = [3, 1, 4, 0, 2]
```

5. Counting Sort (Sắp xếp đếm)

Khi giá trị nằm trong khoảng nhỏ (ví dụ $0 \leq a_i \leq 10^6$), ta có thể dùng counting sort với độ phức tạp $O(n + \max_val)$, nhanh hơn $O(n \log n)$.

Ý tưởng

1. Đếm số lần xuất hiện của mỗi giá trị.
2. Dựng lại mảng từ bảng đếm.

Ví dụ trực quan

Mảng gốc: [3, 1, 4, 1, 5, 9, 2, 6, 5, 3]

Bước 1: Đếm

Giá trị:	0	1	2	3	4	5	6	7	8	9
Số lần:	0	2	1	2	1	2	1	0	0	1

Bước 2: Dựng lại Lấy 2 số 1, 1 số 2, 2 số 3, 1 số 4, 2 số 5, 1 số 6, 1 số 9

→ [1, 1, 2, 3, 3, 4, 5, 5, 6, 9]

Code

C++

```
const int MAX_VAL = 1000000;
int cnt[MAX_VAL + 1] = {};
// Bước 1: Đếm
for (int i = 0; i < n; i++) cnt[a[i]]++;
// Bước 2: Dựng lại
int idx = 0;
for (int v = 0; v <= MAX_VAL; v++) {
    while (cnt[v] > 0) {
        a[idx++] = v;
        cnt[v]--;
    }
}
```

Python

```
MAX_VAL = 10**6
cnt = [0] * (MAX_VAL + 1)
# Bước 1: Đếm
for x in a:
    cnt[x] += 1
# Bước 2: Dựng lại
```

```
res = []
for v in range(MAX_VAL + 1):
    if cnt[v] > 0:
        res.extend([v] * cnt[v])
a = res
```

Khi nào dùng counting sort?

- Giá trị nhỏ ($\leq 10^6$ hoặc 10^7)
- Cần tốc độ tối đa
- Muốn đếm tần suất (không cần dựng lại mảng)

Ghi chú Python: Counting sort trong Python không phải lúc nào cũng nhanh hơn `sort()` built-in, vì `sort()` của Python được tối ưu rất tốt. Chỉ nên dùng counting sort khi miền giá trị nhỏ và thật sự phù hợp.

6. Stable Sort (Sắp xếp ổn định)

Sắp xếp ổn định (stable sort) đảm bảo: nếu hai phần tử có cùng giá trị so sánh, thì thứ tự ban đầu của chúng được giữ nguyên.

Ví dụ: Sắp xếp sinh viên theo điểm. Nếu cùng điểm, muốn giữ thứ tự nhập vào:

C++

```
// sort() KHÔNG đảm bảo stable
sort(v.begin(), v.end());
// stable_sort() đảm bảo stable
stable_sort(v.begin(), v.end());
```

Python

```
# sort() của Python là stable
v.sort()
# sorted(v) cũng stable
w = sorted(v)
```

- Trước: An(85), Binh(92), Cuc(85), Dung(90)
- Sort theo điểm giảm dần:
 - `sort()` của C++: có thể ra Binh(92), Dung(90), Cuc(85), An(85)
 - `stable_sort()` của C++: chắc chắn ra Binh(92), Dung(90), An(85), Cuc(85)
 - `sort()` của Python: giữ ổn định như `stable_sort`

7. Loại bỏ phần tử trùng lặp

Một kỹ thuật rất hay dùng: sort rồi loại bỏ trùng.

C++

```
vector<int> a = {3, 1, 4, 1, 5, 9, 2, 6, 5, 3};
sort(a.begin(), a.end());
// a = [1, 1, 2, 3, 3, 4, 5, 5, 6, 9]
a.erase(unique(a.begin(), a.end()), a.end());
// a = [1, 2, 3, 4, 5, 6, 9]
```

Python

```
a = [3, 1, 4, 1, 5, 9, 2, 6, 5, 3]
a.sort()
# a = [1, 1, 2, 3, 3, 4, 5, 5, 6, 9]
res = []
for x in a:
    if not res or res[-1] != x:
        res.append(x)
a = res
# a = [1, 2, 3, 4, 5, 6, 9]
```

Lưu ý:

- Nếu không cần tự cài đặt quá trình loại trùng, một cách ngắn gọn là dùng cấu trúc set tập hợp rồi sắp xếp lại.
- Tuy nhiên, cách duyệt sau khi sort giúp bạn hiểu rõ hơn ý tưởng của thuật toán.

8. Ví dụ chi tiết: Bài "Chênh lệch nhỏ nhất" (Closest Pair)

Đề bài: Cho mảng a gồm n số nguyên. Tìm chênh lệch nhỏ nhất giữa hai phần tử bất kỳ trong mảng.

Giới hạn: $2 \leq n \leq 2 \times 10^5$.

Ví dụ: $a = [7, 2, 9, 4, 1]$. Các cặp chênh lệch: $|7 - 2| = 5$, $|7 - 9| = 2$, $|2 - 4| = 2$, $|4 - 1| = 3$, $|2 - 1| = 1$, ... Đáp án: 1.

Phân tích

- **Cách "ngây thơ":** Thử tất cả cặp $(i, j) \rightarrow O(n^2)$. Với $n = 2 \times 10^5$, quá chậm!
- **Ý tưởng với sort:** Sau khi sort mảng tăng dần, hai phần tử gần nhau nhất về giá trị chắc chắn nằm kề nhau trong mảng đã sort. Vì vậy chỉ cần duyệt các cặp liên kề!

Chạy tay

$a = [7, 2, 9, 4, 1]$

- **Bước 1:** Sort $\rightarrow [1, 2, 4, 7, 9]$
- **Bước 2:** Duyệt các cặp liên kề:

$$|a[1] - a[0]| = |2 - 1| = 1$$

$$|a[2] - a[1]| = |4 - 2| = 2$$

$$|a[3] - a[2]| = |7 - 4| = 3$$

$$|a[4] - a[3]| = |9 - 7| = 2$$

- **Đáp án:** $\min(1, 2, 3, 2) = 1$.

Code

C++

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n;
    cin >> n;
    vector<int> a(n);
    for (int &x : a) cin >> x;
    sort(a.begin(), a.end());
    int min_diff = INT_MAX;
    for (int i = 1; i < n; i++) {
        min_diff = min(min_diff, a[i] - a[i - 1]);
    }
    cout << min_diff << "\n";
}
```

Python

```
n = int(input())
a = list(map(int, input().split()))
a.sort()
min_diff = 10**18
for i in range(1, n):
    min_diff = min(min_diff, a[i] - a[i - 1])
print(min_diff)
```

Độ phức tạp: $O(n \log n)$ cho sort + $O(n)$ cho duyệt = $O(n \log n)$. So với $O(n^2)$ ban đầu — nhanh hơn rất nhiều!

Bài học rút ra: Sort biến bài toán "so sánh mọi cặp" thành "chỉ cần xét cặp liên kề"— đây là pattern rất phổ biến.

9. Các lỗi phổ biến khi sắp xếp

LỖI	GIẢI THÍCH	CÁCH TRÁNH
Dùng <code><=</code> trong comparator (C++)	Vi phạm strict weak ordering $\rightarrow$ UB	Luôn dùng <code><</code> hoặc <code>></code>
Quên sort trước khi dùng <code>lower_bound</code> / binary search	Chỉ đúng trên mảng đã sort	Luôn sort trước
Sort sai kiểu (<code>int</code> vs <code>long long</code>)	Giá trị lớn bị tràn khi so sánh	Dùng đúng kiểu dữ liệu
Quên rằng sort thay đổi mảng gốc	Mất thông tin vị trí ban đầu	Dùng index sort hoặc copy mảng
Dùng bubble sort tự viết	Quá chậm $O(n^2)$	Luôn dùng <code>sort()</code> / <code>.sort()</code>
Nhầm giữa <code>sorted(a)</code> và <code>a.sort()</code> trong Python	Một cái tạo list mới, một cái sửa tại chỗ	Dùng đúng theo mục đích

Ghi chú Python: `sorted()` vs `.sort()`

```
a = [3, 1, 2]
b = sorted(a) # b = [1, 2, 3], a vẫn là [3, 1, 2]
a.sort() # a trở thành [1, 2, 3]
```

- `sorted(a)` trả về list mới
- `a.sort()` sắp xếp ngay trên list hiện tại
- Trong CP, nếu không cần giữ mảng gốc, `a.sort()` thường tiện hơn

Tóm tắt bài học

HÀM / KỸ THUẬT	C++	PYTHON	ĐỘ PHỨC TẠP
Sắp xếp tăng dần	<code>sort(begin, end)</code>	<code>a.sort()</code> / <code>sorted(a)</code>	$O(n \log n)$
Sắp xếp theo tiêu chí riêng	<code>sort(begin, end, comp)</code>	<code>sort(key=...)</code>	$O(n \log n)$
Sắp xếp ổn định	<code>stable_sort(begin, end)</code>	<code>sort()</code> đã stable	$O(n \log n)$
Counting sort	Tự cài đặt	Tự cài đặt	$O(n + \max_val)$
Loại bỏ trùng lặp	<code>sort + unique + erase</code>	<code>sort</code> rồi duyệt / <code>sorted(set(a))</code>	$O(n \log n)$
Index sort	<code>sort</code> trên mảng chỉ số	<code>sort</code> trên list chỉ số	$O(n \log n)$

Bây giờ hãy thử sức với các bài tập bên dưới! Bắt đầu từ các bài sort cơ bản, rồi tiến đến bài cần kết hợp sort với các kỹ thuật khác nhé.